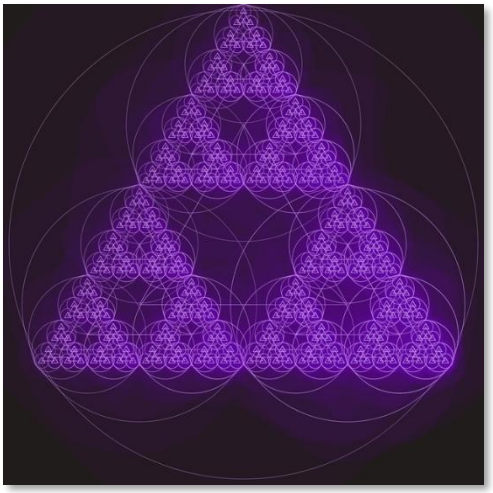
2019



Tema 9: Temario

RECURSIVIDAD

STUDIUM

9.1 Introducción

Ahora vamos a ver la **Recursividad**. Se trata de una forma de programación interrelacionada con la **iteratividad** que ya hemos visto en forma de bucles PARA y bucles MIENTRAS en temas anteriores.

Con la realización de los ejemplos y los ejercicios del presente tema se pretende reconocer la naturaleza recursiva de un problema para aplicar dicha técnica a diferencia de la iterativa.

La recursividad consiste en que una Función o Procedimiento, se llame a sí mismo, pero enviándose un problema de menor complejidad.

Se dice que una función es recursiva cuando se define en función de sí misma.

No todas las funciones pueden llamarse a sí mismas, deben estar diseñadas especialmente para que sean recursivas, de otro modo podrían conducir a bucles infinitos, o a que el programa termine inadecuadamente. Los problemas que resuelven las funciones o procedimientos recursivos son los de cálculo.

Los problemas que se resuelven de forma recursiva deben tener una serie de características que lo permitan. Debido a la complejidad de la Recursividad, no se hace un uso intensivo de ella. Solamente se usará en los casos claros de recursividad, aunque como veremos más adelante, todo problema recursivo se puede resolver de otra forma no recursiva o iterativa.

Dentro de la Recursividad nos encontraremos con el **Caso Base** (o Casos Bases) y los **Casos Recursivos**. Los Casos Bases consisten en la resolución del problema en su caso más sencillo. Los Casos Recursivos, son llamadas a la propia función o procedimiento, pero con unos parámetros simplificados.

La clave de funcionamiento es que obligatoriamente debe existir una Caso Base con el objeto de que la función se bifurque hacia una resolución no recursiva en algún punto. De lo contrario, la función entra en un bucle infinito y nunca finaliza.

Veamos un ejemplo calculando el Factorial de un número natural:

Definición:

factorial (n) = n! = n*n-1*n-2* ... * 1 si n > 0

El valor de 0! se define como factorial (0) = 1

Su definición recursiva es:
factorial (n) = 1 si n = 0 (Caso Base)

factorial (n) = n* factorial (n-1) si n > 0 (Caso Recursivo)

Así para calcular el factorial de 4:

factorial (4) = 4 * factorial (3) = 4 * 6 = 24

factorial (3) = 3 * factorial (2) = 3 * 2 = 6

factorial (2) = 2 * factorial (1) = 2 * 1 = 2

factorial (1) = 1 * factorial (0) = 1 * 1 = 1

9.2 Iteratividad y Recursividad

Para todo problema Recursivo, existe su equivalente en No Recursivo o también llamado Iterativo. De esta forma, podríamos pasar de una solución recursiva y compleja, a una solución iterativa menos complicada. En el caso del factorial visto anteriormente, podemos usar un bucle PARA y obtendríamos el mismo resultado:

```
LEER n
f = 1
PARA i = 1 HASTA n HACER
        f = f * i
FIN PARA
```

La forma de trabajar con la recursividad es siempre siguiendo los pasos que se indican a continuación:
1º Reconocer la naturaleza recursiva de un problema. La resolución de un valor se debe poder averiguar con la simplificación de dicho valor, bien por menor valor, bien por alguna combinación de valores inferiores al buscado. En el factorial de n podemos calcular su valor con el factorial de n-1. Ahí reside la simplificación.

2º Plantear el programa principal desde el que se llamará a la función o procedimiento recursivo tal como veremos en los ejemplos.

3º Definir la función o procedimiento recursivo buscando siempre como mínimo un Caso Base, que será quien rompa la recursividad dando lugar al resultado final y un Caso Recursivo (o varios).

9.3 Ejemplos

Veamos algunos ejemplos. Empezaremos por el **Factorial** de un número dado por teclado.

```
PROGRAMA Factorial
        VARIABLES
                ENTERO n, resultado
        INICIO
                ESCRIBIR "Dame el número n para calcular su factorial:"
                LEER n
                resultado = factorial (n)
                ESCRIBIR resultado
        FIN
        FUNCION factorial(ENTERO a) DEVUELVE ENTERO
                VARIABLES
                        ENTERO r
                INICIO
                        SI a = 0 ENTONCES
                                r = 1 // CASO BASE
                        SINO
                                r = a * factorial(a – 1) // CASO RECURSIVO
                        FIN SI
                        DEVOLVER r
                FIN
FIN PROGRAMA
```

Para la solución en JAVA, voy a colocar una serie de mensajes para que nos salga por pantalla lo que está pasando dentro del programa:

```
package es.studium.Factorial;

import java.util.Scanner;

public class Factorial
{
        public static void main(String[] args)
        {
                int n, resultado;
                Scanner teclado = new Scanner(System.in);
                System.out.println("Dame el número n para calcular su factorial:");
                n = teclado.nextInt();
```

```
            resultado = factorial(n);
            System.out.println("El factorial de "+n+" es:"+resultado);
            teclado.close();
        }
        public static int factorial(int a)
        {
            int r;
            // CASO BASE
            if(a==0)
            {
                r = 1;
                System.out.println("Caso BASE");
            }
            else
            {
                System.out.println("Caso recursivo para a = "+a);
                r = a * factorial(a-1);
                System.out.println("Resultado para a = "+a+" con valor parcial "+r);
            }
            return r;
        }
}
```

En pantalla quedaría algo tal que así:

```
Problems  @ Javadoc  Declaration  Console  Tasks
<terminated> Factorial [Java Application] C:\Program Files (x86)\Java\jre7\bin\javaw.exe (19/02/2016 21:42:51)
Dame el número n para calcular su factorial:
7
Caso recursivo para a = 7
Caso recursivo para a = 6
Caso recursivo para a = 5
Caso recursivo para a = 4
Caso recursivo para a = 3
Caso recursivo para a = 2
Caso recursivo para a = 1
Caso BASE
Resultado para a = 1 con valor parcial 1
Resultado para a = 2 con valor parcial 2
Resultado para a = 3 con valor parcial 6
Resultado para a = 4 con valor parcial 24
Resultado para a = 5 con valor parcial 120
Resultado para a = 6 con valor parcial 720
Resultado para a = 7 con valor parcial 5040
El factorial de 7 es:5040
```

Calcular los n primeros términos de la **sucesión** $\frac{n+3}{2}$

```
PROGRAMA Sucesion
    VARIABLES
        ENTERO n
    INICIO
        ESCRIBIR "Dame el valor de n:"
        LEER n
        sucesion(n)
    FIN
    PROCEDIMIENTO sucesion(ENTERO n)
        VARIABLES
        INICIO
            SI n = 0 ENTONCES
                ESCRIBIR 3/2 // CASO BASE
            SINO
                sucesion(n – 1) // CASO RECURSIVO
                ESCRIBIR (n + 3) / 2
            FIN SI
        FIN
FIN PROGRAMA
```

Observar la diferencia de resultado si intercambiamos el orden de las dos últimas instrucciones del procedimiento, las del SINO:

```
PROGRAMA Sucesion
    VARIABLES
        ENTERO n
    INICIO
        ESCRIBIR "Dame el valor de n:"
        LEER n
        sucesion(n)
    FIN
    PROCEDIMIENTO sucesion(ENTERO n)
        VARIABLES
        INICIO
            SI n = 0 ENTONCES
                ESCRIBIR 3/2 // CASO BASE
            SINO
                ESCRIBIR (n + 3) / 2
                sucesion(n – 1) // CASO RECURSIVO
            FIN SI
        FIN
FIN PROGRAMA
```

Realizar en JAVA este ejemplo, con la modificación sugerida final.

Por último, veremos el método de ordenación de tablas (Ver Tema 3) llamado "**Quicksort**". Este método se basa en la táctica de "divide y vencerás", que consiste en ir subdividiendo el array en arrays más pequeños, y ordenar éstos. Para hacer esta división, se toma un valor del array como pivote, y se mueven todos los elementos menores que este pivote a su izquierda, y los mayores a su derecha. A continuación, se aplica el mismo método a cada una de las dos partes en las que queda dividido el array.

Normalmente se toma como pivote el primer elemento de array, y se realizan dos búsquedas: una de izquierda a derecha, buscando un elemento mayor que el pivote, y otra de derecha a izquierda, buscando un elemento menor que el pivote. Cuando se han encontrado los dos, se intercambian, y se sigue realizando la búsqueda hasta que las dos búsquedas se encuentran. Por ejemplo, para dividir el array {21,40,4,9,10,35}, los pasos serían:

1. {21,40,4,9,10,35} → Se toma como pivote el 21. La búsqueda de izquierda a derecha encuentra el valor 40, mayor que el pivote, y la búsqueda de derecha a izquierda encuentra el valor 10, menor que el pivote. Se intercambian.
2. {21,10,4,9,40,35} → Si seguimos la búsqueda, la primera encuentra el valor 40, y la segunda el valor 9, pero ya se han cruzado, así que paramos. Para terminar la división, se coloca el pivote en su lugar (en el número encontrado por la segunda búsqueda, el 9).
3. {9,10,4,21,40,35} → Ahora tenemos dividido el array en dos arrays más pequeños: el {9,10,4} y el {40,35}, y se repetiría el mismo proceso hasta que no haya más intercambios.

La implementación es claramente **recursiva**, y suponiendo el pivote el primer elemento del array, el algoritmo sería:

```
PROGRAMA Quicksort
CONSTANTES
        ENTERO TAM 10
VARIABLES
        ENTERO tabla[TAM]
INICIO
        PARA i = 0 HASTA TAM-1 HACER
                ESCRIBIR "Dame un número entero:"
                LEER tabla[i]
        FIN PARA
        ordenar(tabla, 0, TAM-1)
        ESCRIBIR "La tabla ordenada queda así:"
        PARA i = 0 HASTA TAM-1 HACER
                ESCRIBIR tabla[i]
        FIN PARA
FIN PROGRAMA
```

```
PROCEDIMIENTO ordenar(ENTERO array[TAM], desde, hasta)
VARIABLES
        ENTERO pivote
INICIO
        SI desde < hasta ENTONCES
                pivote=colocar(array, desde, hasta);
                ordenar(array, desde, pivote-1);
                ordenar(array, pivote+1, hasta);
        FIN SI
FIN
FUNCION colocar(ENTERO array[TAM], desde, hasta) DEVOLVER ENTERO
VARIBLES
        ENTERO i, pivote, valor_pivote, temp
INICIO
        pivote = desde
        valor_pivote = array[pivote]
        PARA i = desde + 1 HASTA i <= hasta HACER
                SI array[i] < valor_pivote ENTONCES
                        pivote = pivote + 1
                        temp = array[i]
                        array[i] = array[pivote]
                        array[pivote] = temp
                FIN SI
        FIN PARA
        temp = array[desde]
        array[desde] = array[pivote]
        array[pivote] = temp
        DEVOLVER pivote
FIN
```

13/09/2025